

Module 3 – Data Link layer

Error Detection and Correction – Hamming code, CRC, Checksum - **Flow Control Mechanism** – Sliding window protocol, GoBack – N – Selective repeat - **Multiple access** – Aloha – Slotted Aloha – CSMA – CSMA/CA – CSMA/CD – **IEEE Standards** (IEEE 802.3(Ethernet), IEEE 802.11(WLAN)) – RFID – Bluetooth Standards

Error in Data Communication

- Whenever the data flow from one point to another, they are subjected to unpredictable changes because of interference
- Many factors can alter one or more bits of messages
- Some applications require a mechanism to detect and correct the errors

Types of Errors:

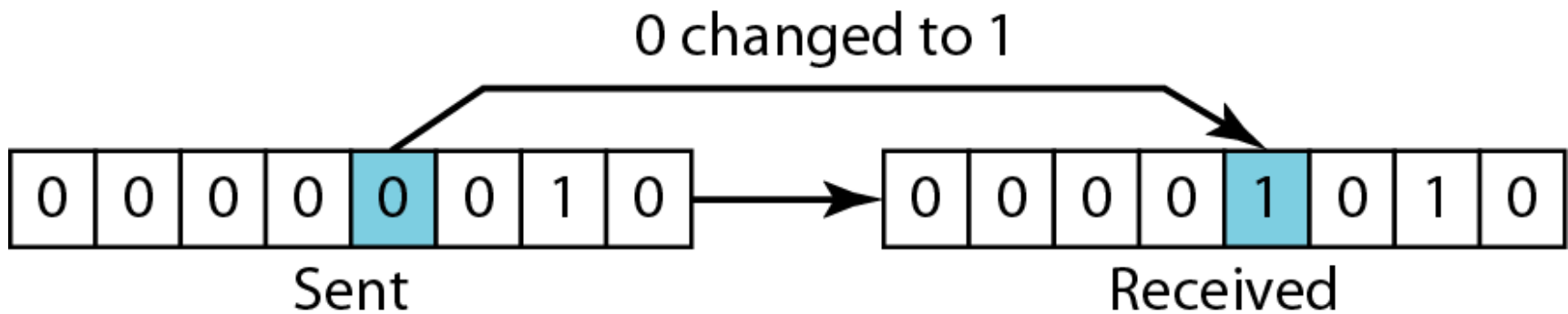
1. Single Bit Error
2. Burst Error



Note

In a single-bit error, only 1 bit in the data unit has changed.

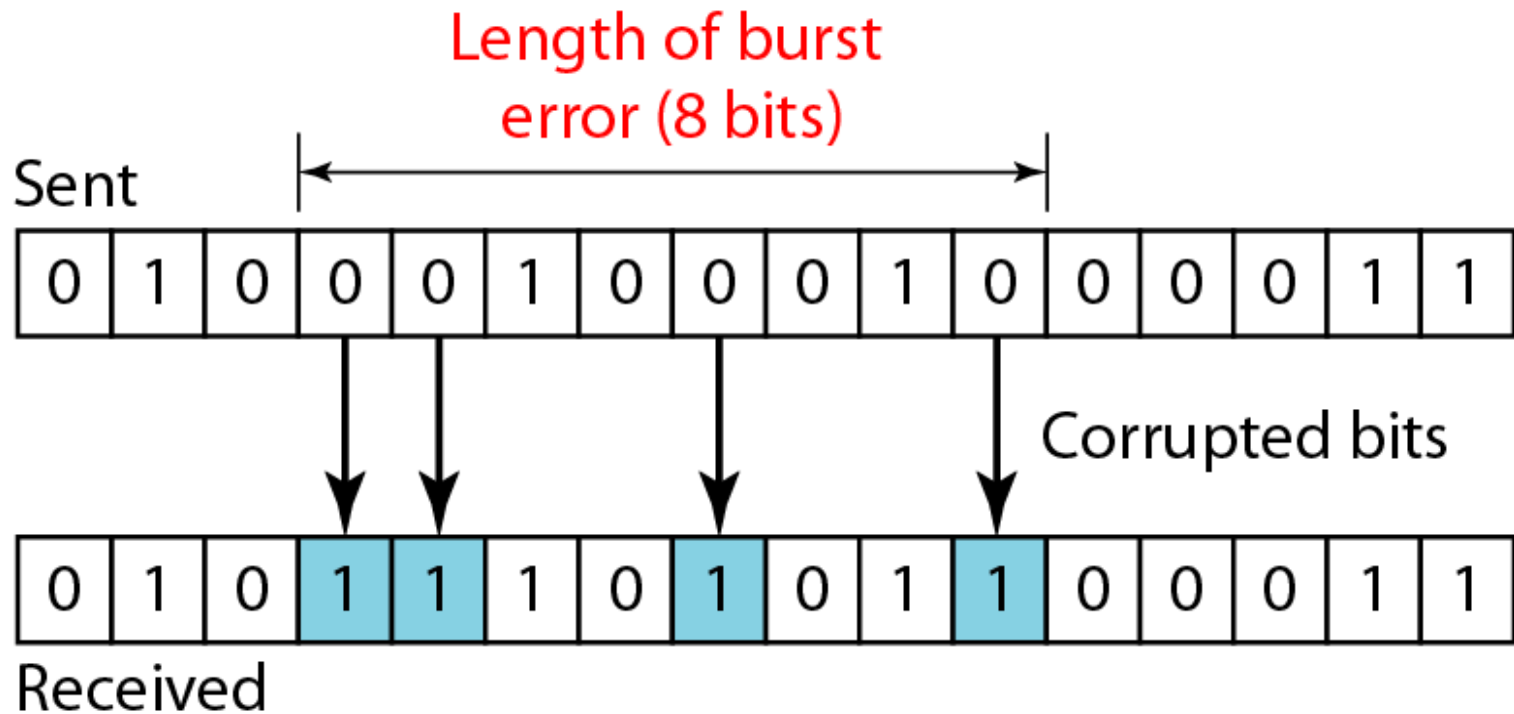
Figure 10.1 *Single-bit error*



Note

A burst error means that 2 or more bits in the data unit have changed.

Figure 10.2 *Burst error of length 8*

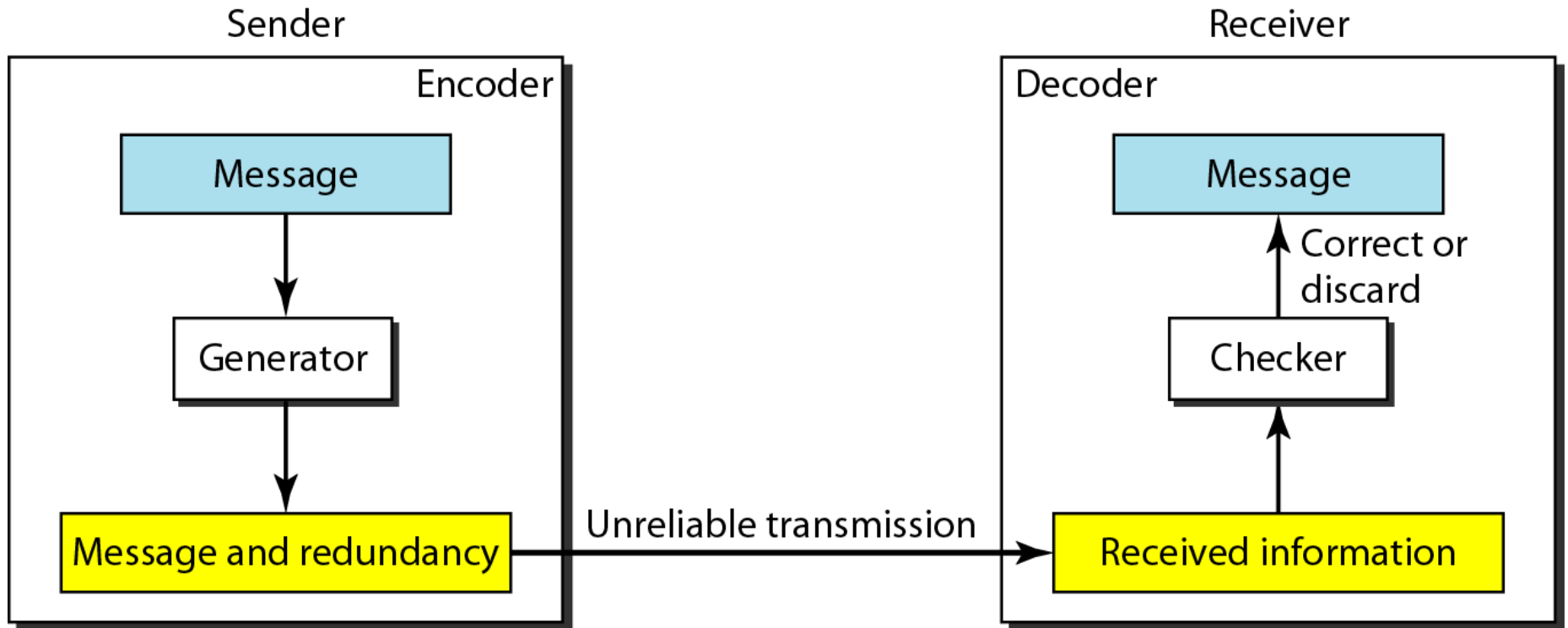




Note

To detect or correct errors, we need to send extra (redundant) bits with data.

Figure 10.3 *The structure of encoder and decoder*

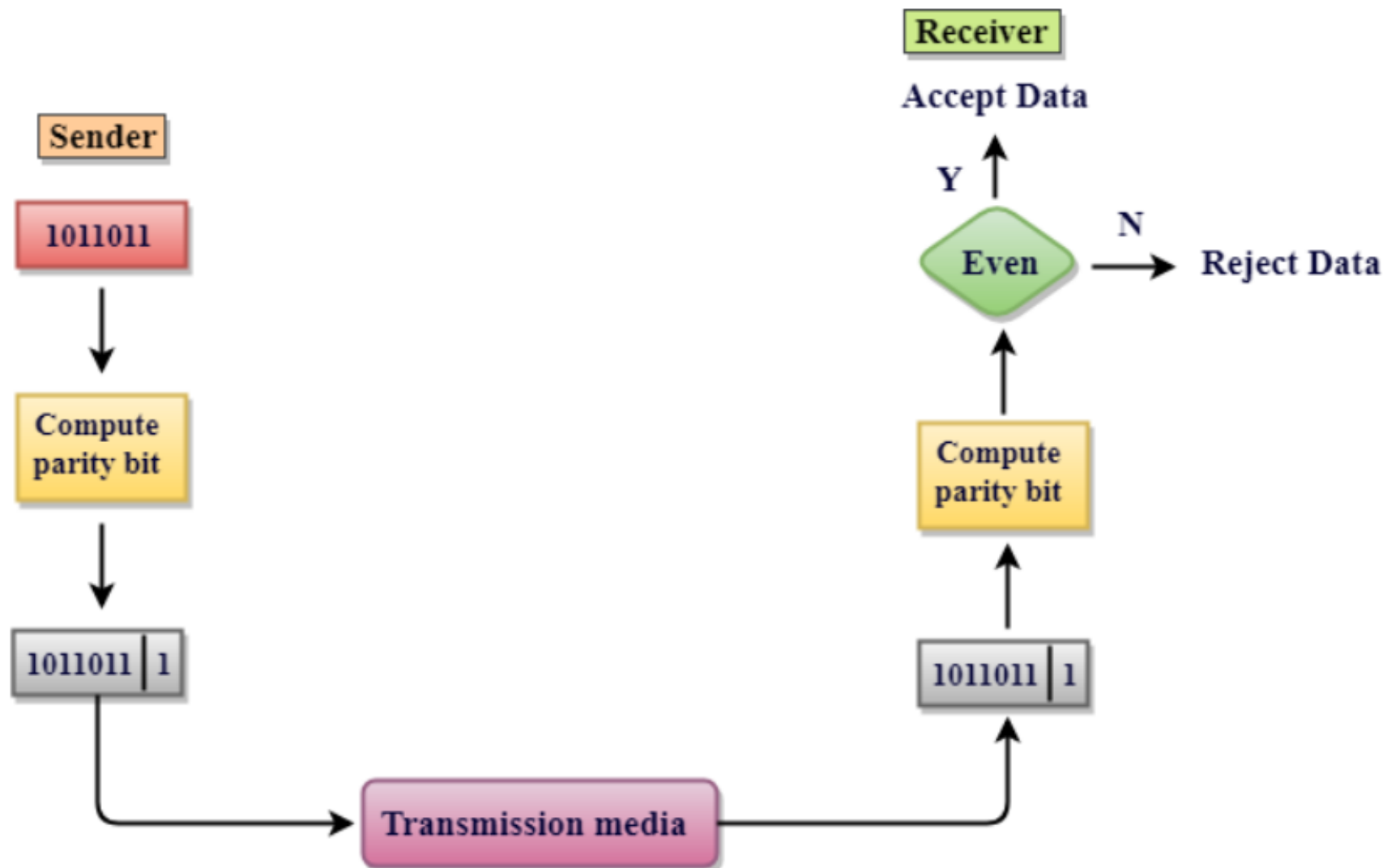


Error Detecting Techniques

- Single parity check
- Two-dimensional parity check
- Checksum
- Cyclic Redundancy Check (CRC)

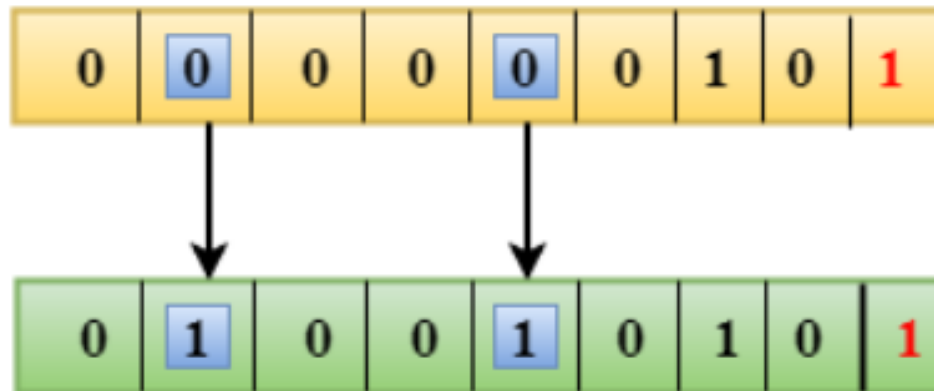
Single Parity Check

- In this technique, a **redundant bit** is also known as **parity bit**.
- Appended at the end of the data unit so that number of 1s become even
- If the **number of 1s bits is odd**, then **parity bit 1** is appended and **if the number of 1s bits is even**, then **parity bit 0** is **appended** at the end of the data unit
- At the receiving end, the parity bit is calculated from the received data bits and compared with the received parity bit



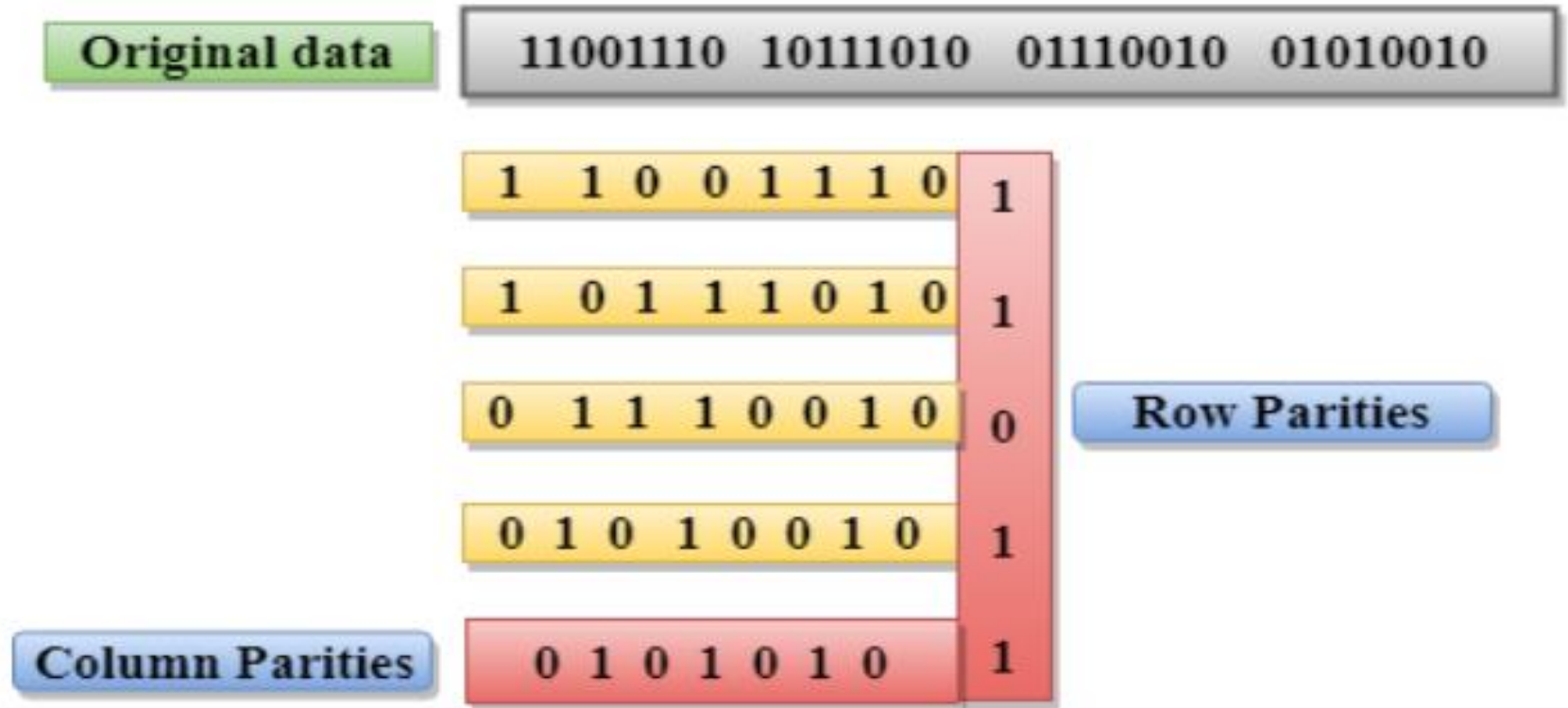
Drawbacks of Single Parity Check

- It can only **detect single-bit errors** which are very rare
- If two bits are interchanged, then it cannot detect the errors.



Two dimensional Parity Check

- Performance can be improved by using Two-Dimensional Parity Check which organizes the data in the form of a table.
- Parity check bits are computed for each row, which is equivalent to the single-parity check.
- In Two-Dimensional Parity check, a block of bits are divided into rows, and the redundant row of bits are added to the whole block.
- At the receiving end, the parity bits are compared with the parity bits computed from the received data.



Data sent: 11001110**1** 10111010**1** 01110010**0** 01010010**1**
010101001****

Drawbacks of 2D Parity Check

- If two bits in one data unit are corrupted and two bits exactly the same position in another data unit are also corrupted, then 2D Parity checker will not be able to detect the error.
- This technique cannot be used to detect the 4-bit errors or more in some cases

Checksum

- It is divided into 2 parts

Checksum generator (At sender side)

Sender site:

- 1. The message is divided into 16-bit words / 8-bit words.**
- 2. The value of the checksum word is set to 0.**
- 3. All words including the checksum are added using one's complement addition.**
- 4. The sum is complemented and becomes the checksum.**
- 5. The checksum is sent with the data.**

■ Checksum Checker(At receiver side)

Receiver site:

- 1.** The message (including checksum) is divided into 16-bit words / 8 bit words.
- 2.** All words are added using one's complement addition.
- 3.** The sum is complemented and becomes the new checksum.
- 4.** If the value of checksum is 0, the message is accepted; otherwise, it is rejected.

Consider the data unit to be transmitted is-

10011001111000100010010010000100

Consider 8 bit checksum is used.

10011001	11100010	00100100	10000100
----------	----------	----------	----------

Now, all the segments are added and the result is obtained as-

1. $10011001 + 11100010 + 00100100 + 10000100 = 1000100011$
2. Since the result consists of 10 bits, so extra 2 bits are wrapped around.
3. $00100011 + 10 = 00100101$ (8 bits)
4. Now, 1's complement is taken which is 11011010.
5. Thus, checksum value = 11011010

At receiver side,

1. The received data unit is divided into segments of 8 bits.
2. All the segments along with the checksum value are added.
3. Sum of all segments + Checksum value = 00100101
+ 11011010 = 11111111
4. Complemented value = 00000000
5. Since the result is 0, receiver assumes no error occurred in the data and therefore accepts it.

10011001011101100001010100001001

Sender:

S1 : 10011001

S2 : 01110110

S3 : 00010101

S4 : 00001001

Sum: 11110011

Sender checksum : 00001100

Sender data : 1001100101110110000101010000100100001100

Received data : 1100110000100011010000000101110000001100

Receiver side calculation:

S1 : 11001100

S2 : 00100011

S3 : 01000000

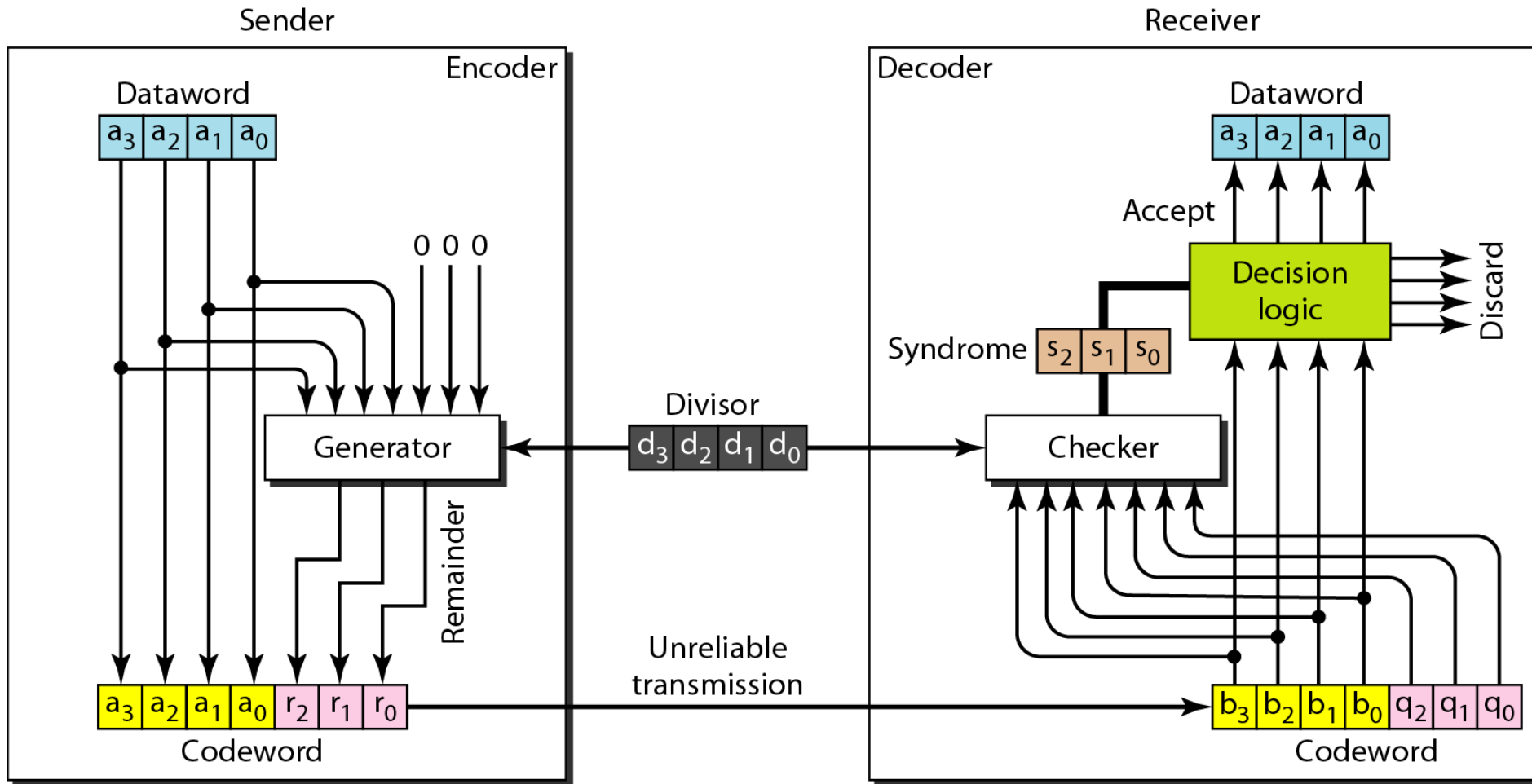
S4 : 01011100

S5 : 01011001

Sum: 10101010 Receiver checksum: 01010101 <=not equal to zero

Cyclic Redundancy Check (CRC)

CRC encoder and decoder



CRC calculation:

A string of **n** as is **appended** to the **data unit**. The length of predetermined divisor is **n+1**.

1. The newly formed data unit *i.e.* **original data + string of n** as are divided by the divisor using binary division and **remainder is obtained**. This **remainder** is called **CRC**.
2. Now, string of n 0s appended to data unit is replaced by the CRC remainder (which is also of n bit).
3. The **data unit + CRC** is then transmitted to **receiver**.
4. The receiver on receiving it **divides data unit + CRC** by the **same divisor** & **checks the remainder**.
5. If the **remainder** of division is **zero**, receiver assumes that there is **no error** in data and it accepts it.
6. If remainder is **non-zero** then there is an **error in data** and receiver rejects it.

Data word : **1001**

Divisor : **1011**

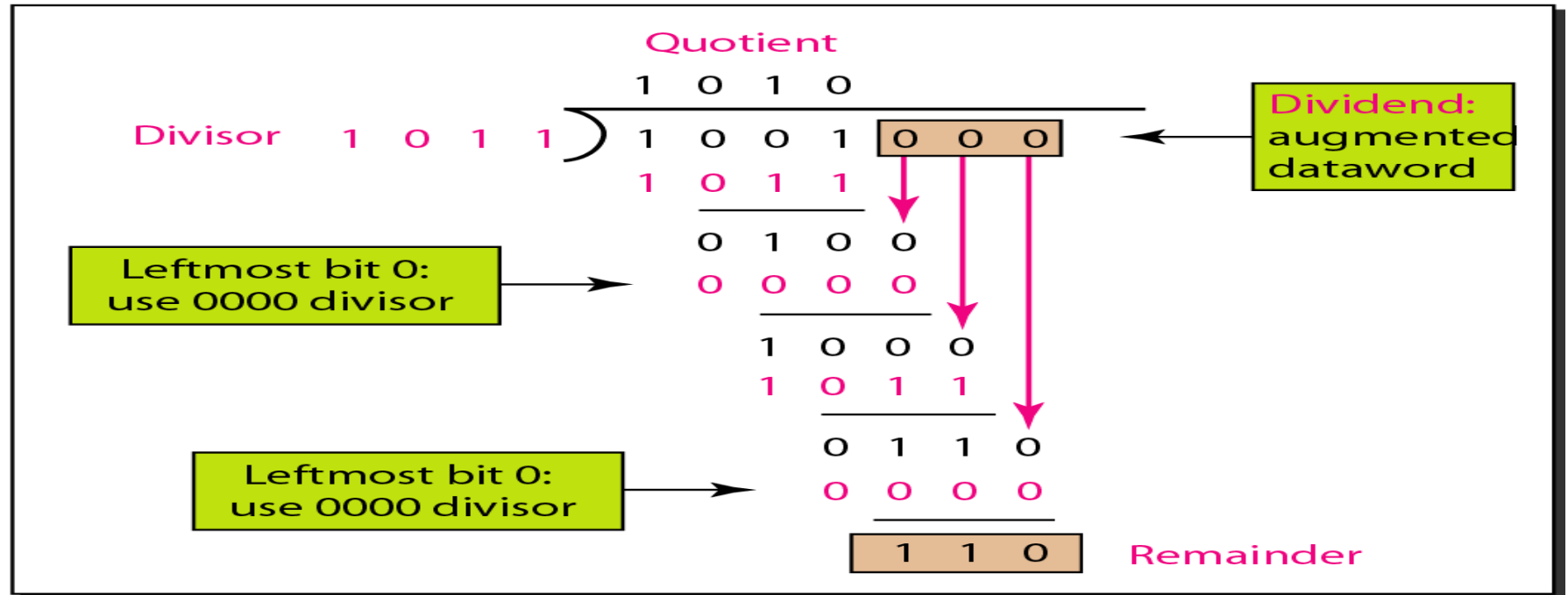
Code word : **Data word + Remainder**

Division in CRC encoder

Dataword

1	0	0	1
---	---	---	---

Division

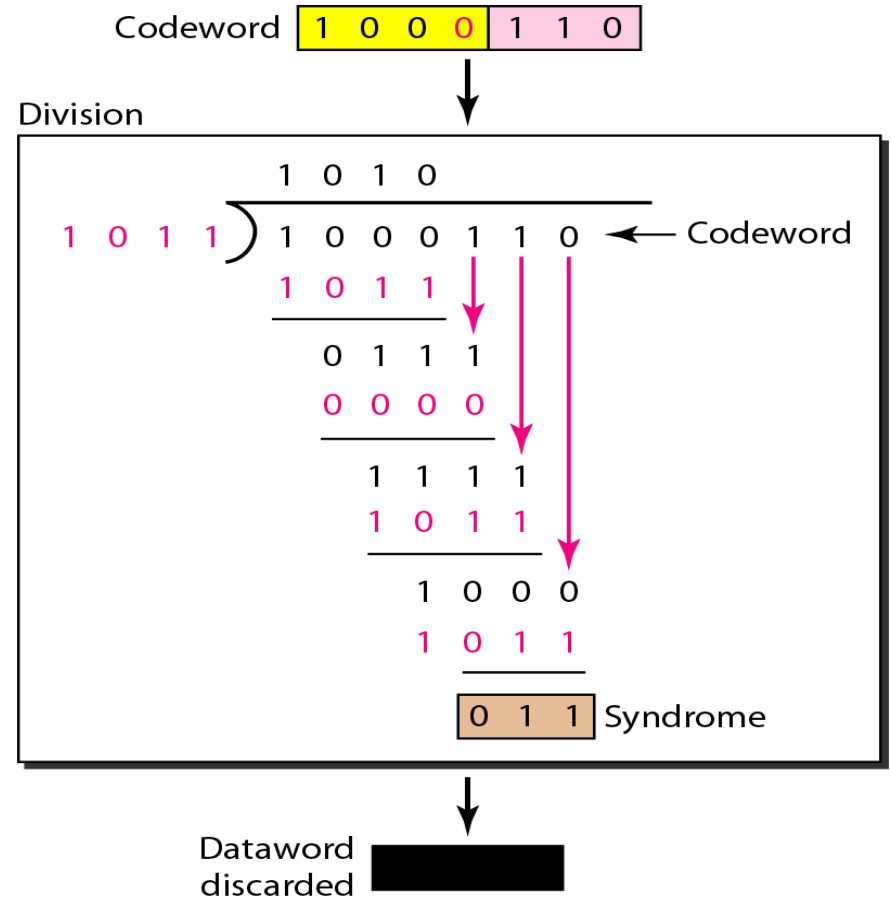
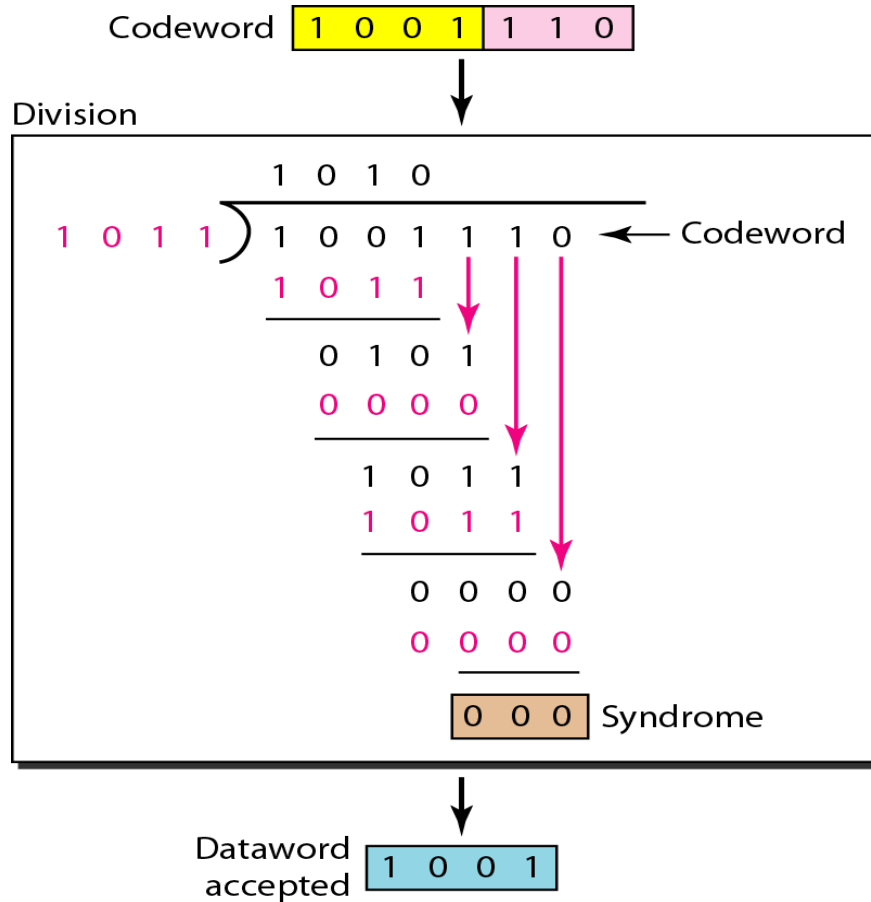


Codeword

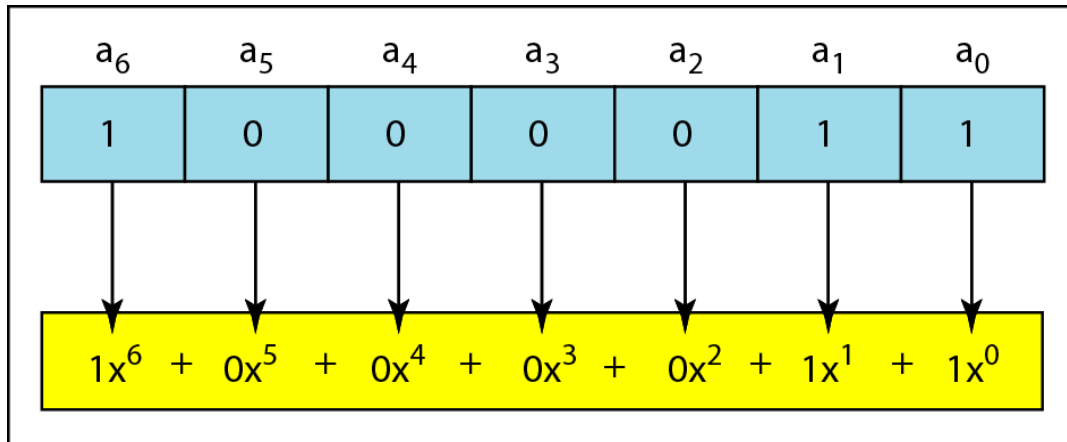
1	0	0	1	1	1	0
---	---	---	---	---	---	---

Dataword Remainder

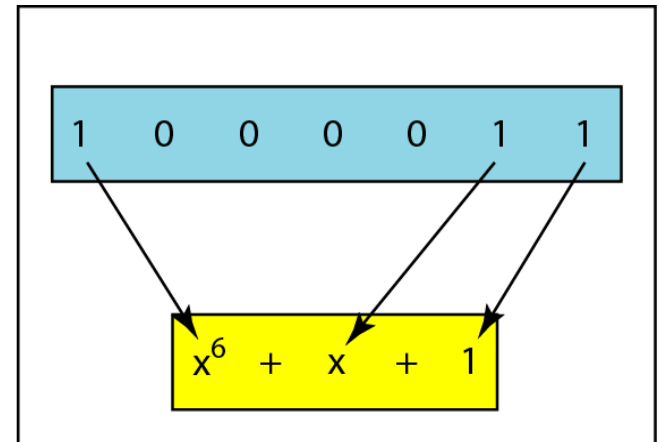
Division in the CRC decoder for two cases



A polynomial to represent a binary word



a. Binary pattern and polynomial



b. Short form

Hamming Code

- Most common types of error-correcting codes used in RAM
- devised by R. W. Hamming.
- In the Hamming code, **k parity bits** are added to an **n-bit data word**, forming a new word of **n + k bits**. The bit **positions** are numbered in sequence from **1 to n + k**.
- **Positions** numbered with **powers of two** are reserved for the parity bits. ($2^0, 2^1, 2^2, 2^3, 2^4 \dots$)
- **Remaining** bits are the **data** bits.

$$2^r \geq m + r + 1$$

where, r = redundant bit, m = data bit

General Algorithm of Hamming code

1. Write the bit positions starting from 1 in binary form (1, 10, 11, 100, etc).
2. All the bit positions that are a **power of 2** are marked as **parity bits** (1, 2, 4, 8, etc).
3. All the **other bit positions** are marked as **data bits**.
4. Each data bit is included in a unique set of parity bits, as determined its bit_position in binary form.
 - a. **Parity bit 1** covers all the bits positions whose binary representation includes a 1 in the least significant position (1,3,5,7,9,11, etc).
 - b. **Parity bit 2** covers all the bits positions whose binary representation includes a 1 in the second position from the least significant bit (2, 3, 6, 7, 10, 11, etc).
 - c. **Parity bit 4** covers all the bits positions whose binary representation includes a 1 in the third position from the least significant bit (4–7, 12–15, 20–23, etc).
 - d. **Parity bit 8** covers all the bits positions whose binary representation includes a 1 in the fourth position from the least significant bit bits (8–15, 24–31, 40–47, etc).
 - e. In general, each parity bit covers all bits where the **bitwise AND** of the parity position and the bit position is non-zero.
5. Since we check for even parity set a **parity bit to 1** if the **total number of ones** in the positions it checks is **odd**.
6. Set a **parity bit to 0** if the **total number of ones** in the positions it checks is **even**.

Position	R8	R4	R2	R1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1

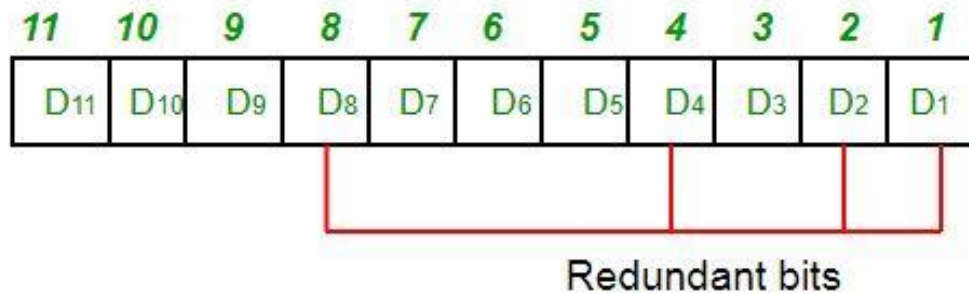
R1 -> 1,3,5,7,9,11
R2 -> 2,3,6,7,10,11
R3 -> 4,5,6,7
R4 -> 8,9,10,11

Determining the position of redundant bits

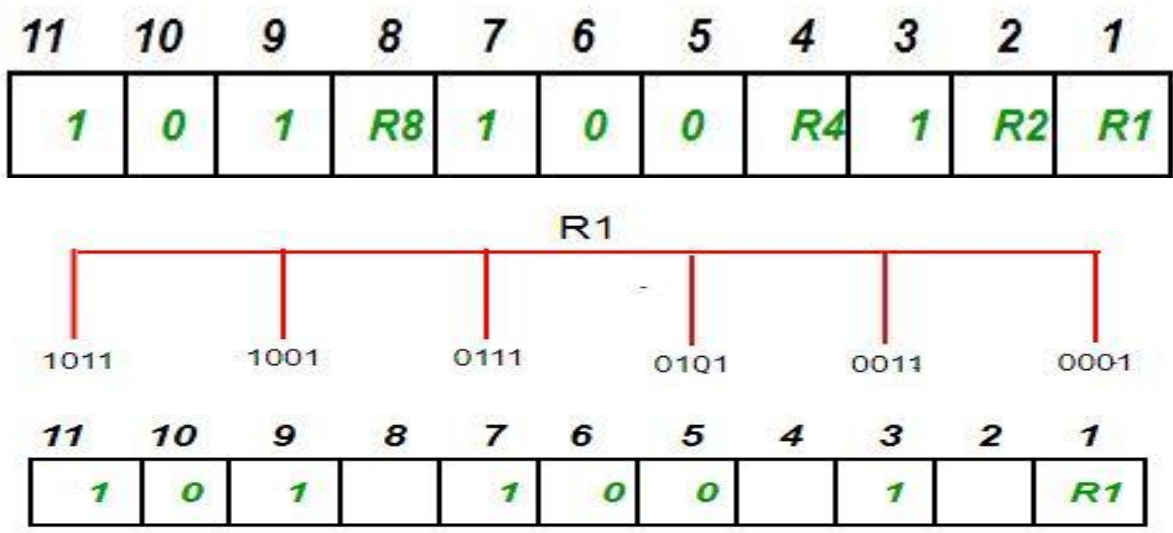
These redundancy bits are placed at the positions which correspond to the power of 2.

As in the above example:

- The number of data bits = 7
- The number of redundant bits = 4
- The total number of bits = 11
- The redundant bits are placed at positions corresponding to **power of 2** → 1, 2, 4, and 8 ($2^0, 2^1, 2^2, 2^3 \dots$)

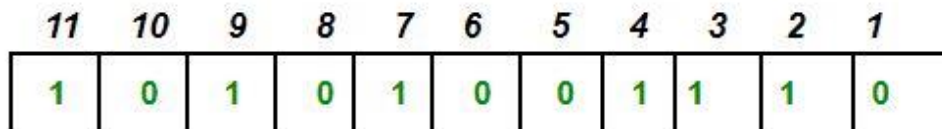
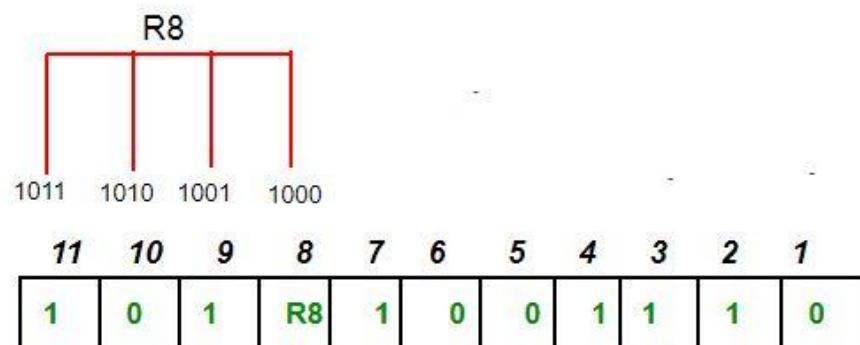
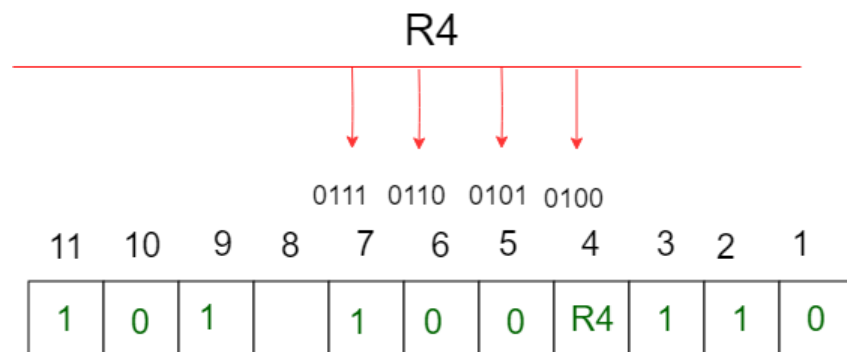
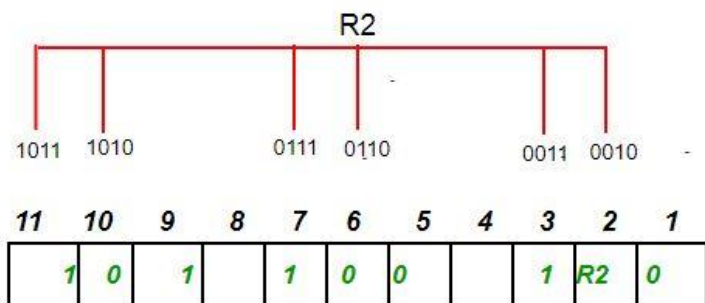


Suppose the data to be transmitted is 1011001, the bits will be placed as follows:

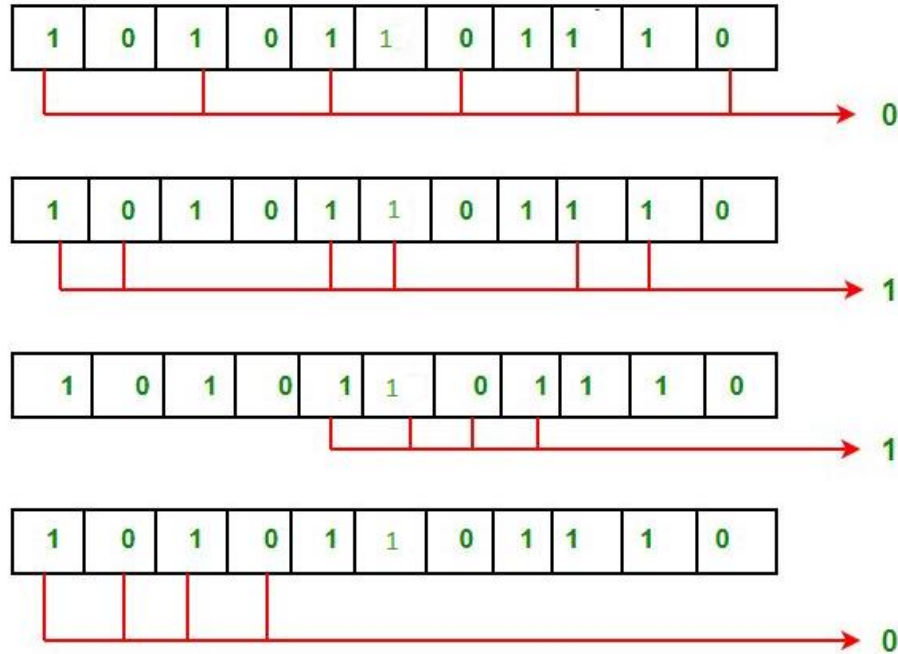


To find the redundant bit R1, we check for even parity. Since the total number of 1's in all the bit positions corresponding to R1 is an even number the value of R1 (parity bit's value) = 0

In the case of even parity, for a given set of bits, the number of 1's are counted. If that count is odd, the **parity bit value is set to 1**, making the total count of occurrences of 1's an even number. If the total number of 1's in a given set of bits is already even, the parity bit's value is 0.



Suppose in the above example the 6th bit is changed from 0 to 1 during data transmission, then it gives new parity values in the binary number:



The bits give the binary number as 0110 whose decimal representation is 6. Thus, the bit 6 contains an error. To correct the error the 6th bit is changed from 1 to 0.